

SeaLabMOS Data Pipeline

Why disregard the built-in salinity values reported by the Atlas Scientific EZO-EC?

The EZO-EC datasheet stipulates that “when you enable the salinity output of the EZO-EC circuit it mathematically converts conductivity to salinity using the Practical Salinity Scale (PSS-78).” [\[source\]](#). It does this, however, using a default temperature of 25°C. The value it uses can be changed, but resets after the system’s power is reset (on each new power-on). This means that, while it is possible in the firmware to send a mid-deployment in-situ temperature to the EZO-EC and chase accurate values “out of the box” from the EZO-EC, we choose to entirely disregard this data. Even following calibration, the salinity (and if enabled the total dissolved solids and specific gravity) values reported by the EZO-EC are incorrect.

We know this because the EZO-EC does not allow a method of changing the pressure used for its supposed PSS-78 calculation (a formula that requires sea pressure), nor does the documentation state what pressure number is used (if any). The calculation is a “black box” and therefore not to be trusted.

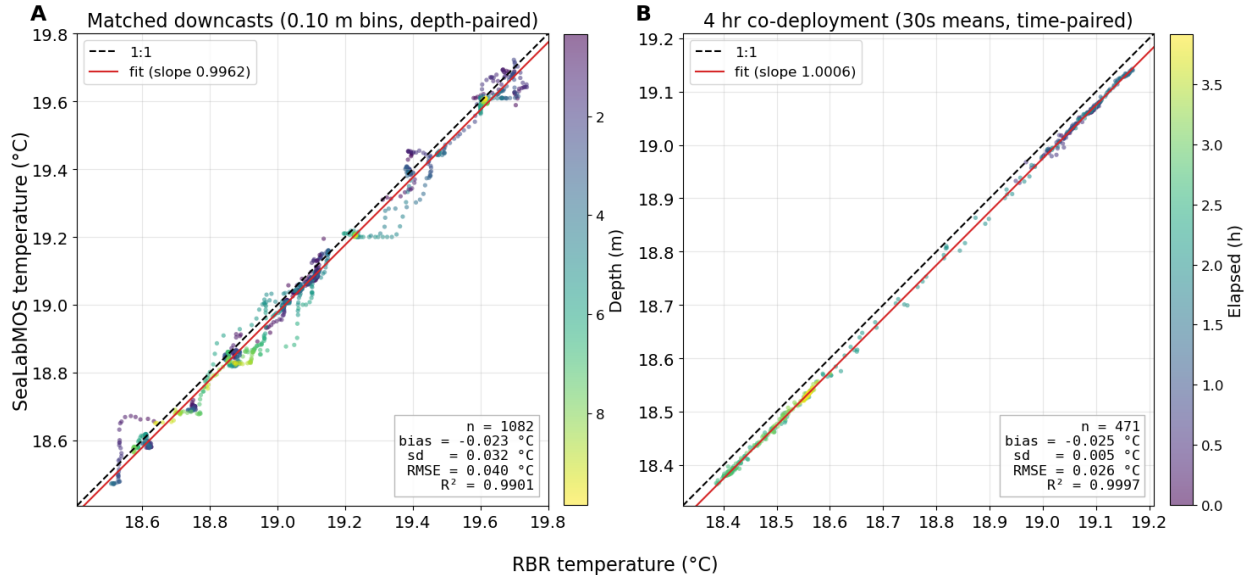
Instead, we opt to do the PSS-78 calculation ourselves from the raw electrical conductivity value as reported by the EZO-EC circuit. This allows full and predictable control over the conversion from EC to salinity using the proper PSS-78 calculation.

The additional benefit of using the probe’s raw in-situ EC value is that the probe’s physical interaction with the sea water is by nature taken at the in-situ temperature of the water. Because we accurately know the temperature of the water from the onboard Blue Robotics Fast Temperature sensor [\[source, source\]](#) and the pressure from the Blue Robotics Pressure Sensor [\[source\]](#), we have all the variables we need: electrical conductivity (in $\mu\text{S}/\text{cm}$ which we convert to mS/cm), temperature, and sea pressure.

$$S_P = \text{SP_from_C}(C_{\text{mS}/\text{cm}}, t_{\text{°C}}, p_{\text{dbar}})$$

A note on temperature accuracy

Per the datasheet, the absolute accuracy of the temperature sensor is $\pm 0.1^\circ\text{C}$ between -5 to 50°C [\[source\]](#). Our results have shown it to be more accurate.



Panel A pools 17 cast-reference pairs collected over 5 days in the same location. Paired casts were separated by a median of 52 s with a max of 5.4 min. Depth rather than time is used as the pairing axis because the instruments were lowered separately, one after the other, meaning they occupied different depths at any given instant.

Panel B is a 4 hour co-location deployment at the same depth in the same location as A's casts. The temperature range reflects either the flood current, surface heating/cooling, or any combination thereof. The cause of temperature change is not material to the accuracy assessment.

Panel B gives a mean offset of -0.025°C with a residual standard deviation of 0.005°C ($n = 471$); Panel A gives a mean offset of -0.023°C with an RMSE of 0.04°C ($n = 1,082$). The worst-case 0.04°C is adopted as the working accuracy figure. Sixteen of seventeen casts show a negative bias, and the two panels agree on its magnitude to within 0.002°C despite using independent pairing methods. The residual scatter was investigated but not isolated to a single cause.

If we propagate the 0.04°C into the practical salinity calculation at fixed conductivity it corresponds to 0.005 PSU at 5 PSU, 0.014 at 15, 0.024 at 25, and 0.031 at 33. This is three to twenty times below the platform's corrected salinity accuracy of ± 0.1 PSU. Thus, no temperature correction is applied at any stage; the variable brFastTemp is passed to gsw.SP_from_C uncorrected.

Data Pipeline and Variable Names Used

SECTION STILL UNDER CONSTRUCTION

MIS_1 correction in the lab looks quadratic while MIS_2 in the field looked scalar. Given what we know so far, I propose a two pipeline data method before calculating SP_from_C, depending on how the system performs: 1) scalar gain like $C_{\text{corrected}} = EC_GAIN \times (ec / 1000)$ and 2) $C = ec / 1000$, then $C_{\text{true}} = A1 \times C + A2 \times C^2$

Variable names that I use

sal_fw = raw firmware salinity, trash

sal_recalc = recomputed salinity from raw EC value

sal_recal_sca = final product salinity values after scalar gain correction

sal_recal_qua = final product salinity values after quadratic correction

The default SeaLabMOS CTD's firmware reports three values from the EZO-EC: electrical conductivity (ec), salinity, and total dissolved solids. Only the first is used.

1. Compute practical salinity using the ec values from the Atlas Scientific EZO-EC circuit and the system's in-situ temperature via PSS-78 by utilizing GSW-Python SP_from_C[\[source\]](#)
2. The returned SP value *should* closely track the RBR, but with the signature of a multiplicative conductivity calibration error intrinsic to the specific SeaLabMOS CTD system (different error per system)
3. Compute a least-squares fit of recomputed vs. reference salinity over the calibration points to find the EC_GAIN correction
4. Apply the EC_GAIN correction factor to the original ec data, then recompute using GSW SP_from_C for final salinity values